Projektanforderungen OOPGames – Kurzfassung

Programmieren 3 · Wintersemester 2026/27 · Josef Griesbauer · ausführliche Fassung: `Projektanforderungen_OOPGames.pdf`

Alle Gruppen arbeiten am WPF-Framework `OOPGames` (C#, .NET 6). Jedes Spiel besteht aus den austauschbaren Rollen **Painter** (`IPaintGame`), **Rules** (`IGameRules`, `IGameField`) und **Player** (`IHumanGamePlayer`, `IComputerGamePlayer`). Die Kompatibilität wird über `CanBePaintedBy` / `CanBeRuledBy` immer gegen *Interfaces* geprüft – Muster: Beispiel `X_Griesbauer`.

Generelle Anforderungen

- **Eigene Verzeichnisse:** Jede Gruppe arbeitet ausschließlich in `Classes/<Gruppe>/` und `Interfaces/<Gruppe>/` (analog `X_Griesbauer`). Dateien: `Vorname_Nachname_<Inhalt>.cs`; Klassen/Interfaces mit Gruppenpräfix; eindeutige `Name`-Properties. Außerhalb davon ist nur das Registrieren der eigenen Klassen im Block `//REGISTER YOUR CLASSES HERE` in `MainWindow.xaml.cs` erlaubt.
- **Jedes Teammitglied committet und pusht selbst** (eigener GitHub-Account) – die Historie zeigt die Aufteilung im Team. Kleine Commits mit aussagekräftigen Messages; vor dem Push `git pull` und `dotnet build`; kein `--force`, keine fremden Dateien, kein `bin/` / `obj/`.
- **Prompts dokumentieren:** Für jeden Studierenden eine eigene Datei `Vorname_Nachname_Prompts.md` im Hauptverzeichnis der Gruppe (`Classes/<Gruppe>/`) mit jedem Prompt im Wortlaut, Datum, Werkzeug/Modell und kurzem Ergebnisvermerk.
- **Pläne dokumentieren:** Alle Pläne (vom Agenten erzeugt oder eigene) als Markdown in `Classes/<Gruppe>/Plans/`. Prompts und Pläne werden zusammen mit dem zugehörigen Code committet.

> **Framework-Dateien sind tabu – auch für Agenten.** Die Basis-Interfaces in `Interfaces/`, `Classes/OOPGames.cs`, `MainWindow.*` (außer Registrierung), `App.*`, `.csproj` / `.sln`, `.vscode/`, `X_Griesbauer` und fremde Gruppenordner werden nicht verändert. Ist eine Framework-Erweiterung nötig, MUSS sie vorab mit dem Dozenten abgeklärt werden. Der Agent ist entsprechend anzuweisen („Ändere nur Dateien in Classes/<Gruppe>/ und Interfaces/<Gruppe>/; ist eine Framework-Änderung nötig, brich ab und beschreibe sie."); vor jedem Commit mit `git status` / `git diff` prüfen.

Anforderung 1: Vier gewinnt – kompatibel über alle Gruppen

- Jede Gruppe implementiert **Painter, Rules (mit Spielfeld), Human-Player und Computer-Player**. Jede Kombination aus Komponenten *aller* Gruppen muss im Hauptfenster funktionieren.
- Grundlage ist ein **gemeinsamer Satz Vier-gewinnt-Interfaces** (z. B. `Interfaces/C4_Gemeinsam/`: `IC4_Field`, `IC4_Move : IColumnMove`, `IC4_Rules`, `IC4_Painter`, `IC4_HumanPlayer`, `IC4_ComputerPlayer`), von allen Gruppen gemeinsam erarbeitet (je eine Vertretung), vom Dozenten freigegeben und danach eingefroren. Teil der Vereinbarung: Feldgröße, Indexkonventionen, Zeichengeometrie (für die Klick-Umrechnung).
- Programmiert wird nur gegen die gemeinsamen Interfaces – nie gegen konkrete Klassen (auch nicht die eigenen).
- Rules weisen ungültige Züge ab und erkennen Sieg (waagerecht, senkrecht, diagonal) und Unentschieden; Human-Player liefert bei ungültiger Eingabe `null`; Computer-Player spielt immer gültig und mindestens „gewinnen, wenn möglich – sonst blockieren".
- **Nachweis:** Test mit mindestens drei anderen Gruppen, dokumentiert als Kompatibilitätsmatrix; Fehler in fremden Komponenten als Issue melden, nicht selbst ändern.

Anforderung 2: Eigenes komplexes Spiel

- Jede Gruppe entwickelt mit Hilfe von Agenten ein **eigenes, komplexes bzw. innovatives Spiel**; Kompatibilität mit anderen Gruppen ist nicht nötig. Erwartet wird deutlich mehr als ein einfacher Brettspielklon (z. B. Echtzeit über `IGameRules2` / `IPaintGame2`, starker KI-Gegner, Levels/Phasen).
- Spielkonzept (eine Seite in `Plans/`) **vor Beginn** dem Dozenten vorstellen.

- **Harte Vorgabe:** Das Spiel läuft vollständig im vorgegebenen Framework – Auswahl und Start über das Hauptfenster, Eingaben nur über Klick-/Tasten-Selections, Zeichnen nur auf das übergebene Canvas, kein eigenes Fenster oder eigene Hauptschleife. Framework-Erweiterungen nur nach Absprache (siehe oben).
- Sauberer OO-Entwurf; das Team muss **den gesamten Code erklären können**, auch den agentisch erzeugten.
- **Teamarbeit nach SCRUM**, dokumentiert in `Classes/<Gruppe>/Scrum/` (oder GitHub Issues/Projects): Product Owner und Scrum Master benannt, Product Backlog mit User Stories, Sprints (1–2 Wochen) mit Sprintziel und Verantwortlichen je Aufgabe, Kurzprotokolle der Abstimmungen, Review und Retrospektive je Sprint. Backlog, Prompt-Dateien, Pläne und Commits müssen zusammenpassen; jedes Mitglied hat einen substanziellen, sichtbaren Anteil.

Termine für Konzeptvorstellung, Zwischenstände und Abnahme werden in der Vorlesung bekanntgegeben. Bei Unklarheiten gilt: vorher fragen.